

Helix AI: Frontend and UX System (In-Depth)

Professional single-page modernization experience

Generated: 2026-04-05 01:24:58

This report describes the inner-page UI design decisions, interaction states, and how the frontend supports safe modernization review.

Project Snapshot	Value
Frontend files	4
Frontend LOC	2647
Sample buttons	3
Result tabs	4
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Frontend Philosophy

Simple Explanation

In simple words: keep the user on one page, show clear steps, and never hide risky changes.

Deep Explanation

Frontend currently uses 4 key files with around 2647 non-empty lines.

The UI is designed around a guided flow: analyze, review, modernize, validate.

The layout keeps input editor and result panel side by side for immediate comparison.

Practical Checklist

- Keep one-screen workflow for focus.
- Always show next action guidance.
- Do not replace input automatically without consent.

Code References

```
templates/index.html
static/js/app.js
static/css/styles.css
```

1. Screen Architecture

Simple Explanation

In simple words: header for context, step strip for progress, editor on left, result tabs on right.

Deep Explanation

Header communicates project identity and mode/status badges.

Step cards represent state transitions with active, done, and warning visual semantics.

Result area includes review, diff, output, and report tabs for layered inspection.

Practical Checklist

- Make status badges obvious at glance.
- Keep critical actions near editor context.
- Limit cognitive load by progressive disclosure.

Code References

```
templates/index.html
```

2. Interaction Flow

Simple Explanation

In simple words: user pastes code, analyzes, sees findings, then chooses to modernize.

Deep Explanation

Primary action button changes label and behavior by state: Analyze Code, Modernize Code, Apply Output.

Keyboard shortcut Cmd/Ctrl + Enter triggers next primary action.

Sample snippet buttons reduce friction in demos and viva presentations.

Practical Checklist

- Preserve deterministic button state machine.
- Keep keyboard shortcuts discoverable.
- Provide sample snippets for fast walkthroughs.

Code References

`static/js/app.js`

3. State Management

Simple Explanation

In simple words: a single JavaScript state object tracks code, analysis, plan, diff, report, and logs.

Deep Explanation

State fields include originalCode/newCode/diff/report/runId plus agent outputs.

Rendering functions (renderReview/renderDiff/renderOutput/renderReport) read state and update panels.

This central state model avoids scattered DOM-only logic.

Practical Checklist

- Keep state schema explicit.
- Update state before rendering.
- Avoid hidden side effects inside renderers.

Code References

`static/js/app.js`

4. Diff-First UX

Simple Explanation

In simple words: users inspect changes first, then apply to editor only if they accept them.

Deep Explanation

Diff panel uses visual line classes for added/removed/meta lines.

Output and report remain available without forcing immediate overwrite of editor input.

This reduces accidental acceptance of unsafe changes.

Practical Checklist

- Show diff before apply.
- Keep apply action explicit and reversible via rerun.
- Include warning text when validation is not fully clean.

Code References

`templates/index.html`
`static/js/app.js`

5. Run History UX

Simple Explanation

In simple words: recent runs are clickable so users can re-open previous modernization reports quickly.

Deep Explanation

UI fetches summarized runs then lazy-loads full details when a run is clicked.

Loaded runs repopulate review/diff/output/report panels without modifying current editor text unless user applies.

This interaction supports iterative experimentation safely.

Practical Checklist

- Keep history item compact and scannable.
- Display validation status per run.
- Load full run on click, not upfront.

Code References

`static/js/app.js`
`core/run_store.py`

6. Accessibility and Responsiveness

Simple Explanation

In simple words: UI should still be understandable on smaller screens and keyboard-first usage.

Deep Explanation

Layout degrades from dual-column desktop to stacked mobile flow through responsive classes.

Interactive controls are keyboard reachable and tab sequences are predictable.

Reduced-motion media queries are included to lower animation intensity where needed.

Practical Checklist

- Validate with keyboard-only navigation.
- Keep text contrast high in dark theme.
- Ensure action labels remain clear on mobile widths.

Code References

`templates/index.html`

7. Frontend Maturity Plan

Simple Explanation

In simple words: next UX upgrades are guided walkthroughs, inline explanations, and visual analytics.

Deep Explanation

Add contextual explanation popovers for each detected legacy pattern.

Add per-transform confidence badges directly in review cards.

Add lightweight performance telemetry to detect slow API calls and large-code rendering issues.

Practical Checklist

- Prioritize clarity over visual novelty.
- Keep API error handling user-friendly.
- Preserve current clean single-page mental model.

Code References

`templates/index.html`
`static/js/app.js`

Viva Quick Answers

Q: Why diff-first UX?

Because migration trust depends on seeing exact line-level changes before applying.

Q: How is user flow simplified?

Single page, clear step cards, guided focus text, and one primary action button.

Q: How do you avoid confusion?

Separate Analyze and Modernize phases and keep output in tabs before applying to editor.

Q: How is run history useful?

It allows reopening previous reports and diffs without rerunning the pipeline.

Q: What is next for UI?

Contextual explanations, confidence badges, and richer per-pattern insights.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Single-Page Workflow Rationale

Simple view: single-page workflow rationale explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, single-page workflow rationale connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Information Hierarchy And Layout

Simple view: information hierarchy and layout explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, information hierarchy and layout connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Editor And Results Dual-Panel Strategy

Simple view: editor and results dual-panel strategy explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, editor and results dual-panel strategy connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Primary Action Button State Model

Simple view: primary action button state model explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, primary action button state model connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: Tab Rendering And Result Transparency

Simple view: tab rendering and result transparency explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, tab rendering and result transparency connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: Run History Interaction Pattern

Simple view: run history interaction pattern explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, run history interaction pattern connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: Keyboard Shortcuts And Speed

Simple view: keyboard shortcuts and speed explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, keyboard shortcuts and speed connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: Hover And Motion Behavior

Simple view: hover and motion behavior explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, hover and motion behavior connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: Responsiveness And Mobile Fallback

Simple view: responsiveness and mobile fallback explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, responsiveness and mobile fallback connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: Error Feedback And User Trust

Simple view: error feedback and user trust explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, error feedback and user trust connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: Accessibility Improvements Roadmap

Simple view: accessibility improvements roadmap explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, accessibility improvements roadmap connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: Frontend Instrumentation Roadmap

Simple view: frontend instrumentation roadmap explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, frontend instrumentation roadmap connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to frontend concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in state rendering	Diagram tied to too concrete file and flow	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. State Store

Central object holding UI workflow data.

22. Render Function

Function that converts state into visible UI.

23. Primary Action

Main CTA button behavior by workflow phase.

24. Progressive Disclosure

Show details only when user needs them.

25. Interaction Latency

Delay felt between action and UI response.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: Why keep input and results on same page?

It reduces context switching and makes diff review faster.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: How do keyboard shortcuts help?

They speed up demo and expert workflows.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: What frontend risk remains?

Large-file rendering and complexity can grow without careful UX constraints.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: Inner Page Structure

File: /Users/rutwikbhondave/Desktop/Helix AI Project/templates/index.html

Simple explanation: this block handles UI structure and screen information hierarchy. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=0, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: <!DOCTYPE html>
0002: <html lang="en" class="dark">
0003: <head>
0004:     <meta charset="UTF-8">
0005:     <meta name="viewport" content="width=device-width, initial-scale=1.0">
0006:     <title>Helix AI | Legacy Python Modernization</title>
0007:     <script src="https://cdn.tailwindcss.com"></script>
0008:     <link href="https://fonts.googleapis.com/css2?family=Geist:wght@300;400;500;600;700&display=swap" rel="stylesheet">
0009:     <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
0010:     <script>
0011:         tailwind.config = {
0012:             darkMode: "class",
0013:             theme: {
0014:                 extend: {
0015:                     fontFamily: {
0016:                         sans: ["Geist", "sans-serif"],
0017:                         mono: ["JetBrains Mono", "monospace"],
0018:                     },
0019:                 },
0020:             },
0021:         };
0022:     </script>
0023:     <style>
0024:         body { font-family: "Geist", sans-serif; }
0025:         @keyframes fadeInUp {
0026:             from { opacity: 0; transform: translateY(12px); }
0027:             to { opacity: 1; transform: translateY(0); }
0028:         }
0029:         @keyframes softPulse {
0030:             0%, 100% { box-shadow: 0 0 0 rgba(249, 115, 22, 0); }
0031:             50% { box-shadow: 0 0 24px rgba(249, 115, 22, 0.16); }
0032:         }
0033:         .animate-fade-in-up { animation: fadeInUp 0.25s ease-out forwards; }
0034:         .custom-scrollbar::-webkit-scrollbar { width: 6px; height: 6px; }
0035:         .custom-scrollbar::-webkit-scrollbar-track { background: #18181b; }
0036:         .custom-scrollbar::-webkit-scrollbar-thumb { background: #3f3f46; border-radius: 999px; }
0037:         .step-item,
0038:         .sampleBtn,
```

```

0039: .tab-btn,
0040: #downloadReport,
0041: .copy-btn,
0042: #advancedToggleBtn,
0043: #summaryStrip > div,
0044: #runHistory button,
0045: header a[href="/"],
0046: .feature-pill {
0047:     transition: transform 180ms ease, border-color 180ms ease, background-color 180ms ease;
0048: }
0049: .feature-pill:hover {
0050:     transform: translateY(-1px);
0051:     color: #fff7ed;
0052:     border-color: rgba(249, 115, 22, 0.4);
0053:     background: rgba(249, 115, 22, 0.12);
0054:     box-shadow: 0 10px 22px rgba(249, 115, 22, 0.12);
0055: }
0056: .step-item:hover {
0057:     transform: translateY(-2px);
0058:     border-color: rgba(255, 255, 255, 0.10);
0059:     box-shadow: 0 14px 28px rgba(0, 0, 0, 0.18);
0060: }
0061: .step-item.current .step-dot {
0062:     border-color: rgba(249, 115, 22, 0.7);
0063:     background: rgba(249, 115, 22, 0.16);
0064:     color: #fff7ed;
0065:     box-shadow: 0 0 0 1px rgba(249, 115, 22, 0.16), 0 0 24px rgba(249, 115, 22, 0.12);
0066:     animation: softPulse 2.4s ease-in-out infinite;
0067: }
0068: .step-item.done .step-dot {
0069:     border-color: rgba(16, 185, 129, 0.6);
0070:     background: rgba(16, 185, 129, 0.14);
0071:     color: #d1fae5;
0072: }
0073: .step-item.done .step-label { color: #e5e7eb; }
0074: .step-item.current .step-label { color: #ffffff; }
0075: .step-item.current .step-copy { color: #a1a1aa; }
0076: #runBtn {
0077:     transition: transform 180ms ease, box-shadow 220ms ease, background-color 180ms ease;
0078: }
0079: #runBtn:not(:disabled):hover {
0080:     transform: translateY(-1px);
0081:     box-shadow: 0 14px 28px rgba(255, 255, 255, 0.10), 0 0 18px rgba(249, 115, 22, 0.10);
0082: }
0083: #runBtn:not(:disabled):active {
0084:     transform: translateY(0);
0085:     box-shadow: 0 6px 14px rgba(0, 0, 0, 0.20);
0086: }
0087: .sampleBtn:hover,
0088: .copy-btn:hover,
0089: #downloadReport:hover,
0090: header a[href="/"]:hover {
0091:     transform: translateY(-1px);
0092:     box-shadow: 0 10px 24px rgba(0, 0, 0, 0.18);
0093: }
0094: .tab-btn.active {
0095:     color: white;
0096:     border-bottom: 2px solid rgb(249 115 22);
0097:     background: rgba(255,255,255,0.05);
0098: }
0099: .tab-btn:hover {
0100:     color: #ffffff;
0101:     background: rgba(255,255,255,0.04);
0102: }
0103: #summaryStrip > div:hover,
0104: #runHistory button:hover {
0105:     transform: translateY(-2px);
0106:     border-color: rgba(249, 115, 22, 0.22);
0107:     box-shadow: 0 14px 28px rgba(0, 0, 0, 0.18);

```

```

0108:     }
0109:     #advancedToggleBtn:hover {
0110:         background: rgba(255, 255, 255, 0.03);
0111:     }
0112:     #advancedToggleBtn:hover #advancedToggleLabel {
0113:         color: #f4f4f5;
0114:     }
0115:     .diff-line.add { color: #86efac; background: rgba(34, 197, 94, 0.08); }
0116:     .diff-line.remove { color: #fda4af; background: rgba(244, 63, 94, 0.08); }
0117:     .diff-line.meta { color: #7dd3fc; }
0118:     .panel-shell {
0119:         animation: fadeInUp 0.22s ease-out;
0120:     }
0121:     .step-item button {
0122:         cursor: pointer;
0123:     }
0124:     @media (prefers-reduced-motion: reduce) {
0125:         .animate-fade-in-up,
0126:         .step-item.current .step-dot {
0127:             animation: none !important;
0128:         }
0129:         .step-item,
0130:         .sampleBtn,
0131:         .tab-btn,
0132:         #runBtn,
0133:         #downloadReport,
0134:         .copy-btn,
0135:         #advancedToggleBtn,
0136:         #summaryStrip > div,
0137:         #runHistory button,
0138:         header a[href="/"],
0139:         .feature-pill {
0140:             transition: none !important;

```

Excerpt 2: UI State Machine Part 1

File: /Users/rutwikbhondave/Desktop/Helix AI Project/static/js/app.js

Simple explanation: this block handles frontend state machine and user interaction handling. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=2, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: emit_values, normalize_name.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: const SAMPLE_SNIPPETS = {
0002:   print: `for i in xrange(3):
0003:     print i`,
0004:   dict: `def emit_values(payload):
0005:     if payload.has_key("alpha"):
0006:       for key, value in payload.iteritems():
0007:         print key, value`,
0008:   input: `def normalize_name(name):
0009:     if isinstance(name, basestring):
0010:       return unicode(name).strip()
0011:     return raw_input("fallback: ")`,
0012: };
0013:
0014: const state = {
0015:   originalCode: "",
0016:   newCode: "",
0017:   diff: "",
0018:   report: "",
0019:   runId: "",
0020:   analysis: null,
0021:   suggestions: [],
0022:   critiques: [],
0023:   plan: null,
0024:   validation: null,
0025:   ml: null,
0026:   logs: [],
0027: };
0028:
0029: const codeInput = document.getElementById("codeInput");
0030: const lineNumbers = document.getElementById("lineNumbers");
0031: const reviewPanel = document.getElementById("reviewPanel");
0032: const diffPanel = document.getElementById("diffPanel");
0033: const outputPanel = document.getElementById("outputPanel");
0034: const reportPanel = document.getElementById("reportPanel");
0035: const terminal = document.getElementById("terminal");
0036: const runHistory = document.getElementById("runHistory");
0037: const mlSummary = document.getElementById("mlSummary");
0038: const mlBadge = document.getElementById("mlBadge");
0039: const focusText = document.getElementById("focusText");
0040: const modeBadge = document.getElementById("modeBadge");
0041: const statusBadge = document.getElementById("statusBadge");
0042: const summaryStrip = document.getElementById("summaryStrip");
```

```

0043: const summarySource = document.getElementById("summarySource");
0044: const summaryIssues = document.getElementById("summaryIssues");
0045: const summaryChanges = document.getElementById("summaryChanges");
0046: const summaryState = document.getElementById("summaryState");
0047: const advancedPanel = document.getElementById("advancedPanel");
0048: const advancedToggleBtn = document.getElementById("advancedToggleBtn");
0049: const advancedToggleLabel = document.getElementById("advancedToggleLabel");
0050: const runBtn = document.getElementById("runBtn");
0051: const actionHint = document.getElementById("actionHint");
0052: const downloadReport = document.getElementById("downloadReport");
0053: const copyDiffBtn = document.getElementById("copyDiffBtn");
0054: const copyOutputBtn = document.getElementById("copyOutputBtn");
0055:
0056: const tabs = {
0057:   review: document.getElementById("reviewTab"),
0058:   diff: document.getElementById("diffTab"),
0059:   output: document.getElementById("outputTab"),
0060:   report: document.getElementById("reportTab"),
0061: };
0062:
0063: const panels = {
0064:   review: reviewPanel,
0065:   diff: diffPanel,
0066:   output: outputPanel,
0067:   report: reportPanel,
0068: };
0069:
0070: const steps = {
0071:   analyze: document.getElementById("stepAnalyze"),
0072:   review: document.getElementById("stepReview"),
0073:   modernize: document.getElementById("stepModernize"),
0074:   validate: document.getElementById("stepValidate"),
0075: };
0076:
0077: document.addEventListener("DOMContentLoaded", () => {
0078:   bindEvents();
0079:   resetUi();
0080:   fetchModelStatus();
0081:   loadRunHistory();
0082: });
0083:
0084: function bindEvents() {
0085:   codeInput.addEventListener("input", updateLineNumbers);
0086:   codeInput.addEventListener("scroll", syncScroll);
0087:
0088:   document.querySelectorAll(".sampleBtn").forEach((button) => {
0089:     button.addEventListener("click", () => {
0090:       const sample = SAMPLE_SNIPPETS[button.dataset.sample];
0091:       if (!sample) {
0092:         return;
0093:       }
0094:       codeInput.value = sample;
0095:       updateLineNumbers();
0096:       syncScroll();
0097:       showToast("Sample loaded", "success");
0098:     });
0099:   });
0100:
0101:   document.addEventListener("keydown", (event) => {
0102:     if ((event.metaKey || event.ctrlKey) && event.key === "Enter") {
0103:       event.preventDefault();
0104:       runBtn.click();
0105:     }
0106:   });
0107:
0108:   Object.entries(tabs).forEach(([key, button]) => {
0109:     button.addEventListener("click", () => activateTab(key));
0110:   });
0111:

```

```

0112:     Object.entries(steps).forEach(([key, button]) => {
0113:         button.addEventListener("click", () => handleStepNavigation(key));
0114:     });
0115:
0116:     advancedToggleBtn.addEventListener("click", () => setAdvancedOpen(advancedPanel.classList.contains("active")));
0117:     runHistory.addEventListener("click", handleRunHistoryClick);
0118:     runBtn.addEventListener("click", handlePrimaryAction);
0119:     copyDiffBtn.addEventListener("click", () => copyPanelContent("diff"));
0120:     copyOutputBtn.addEventListener("click", () => copyPanelContent("output"));
0121: }
0122:
0123: async function handlePrimaryAction() {
0124:     const action = state.primaryAction || "analyze";
0125:     if (action === "analyze") {
0126:         await handleAnalyze();
0127:         return;
0128:     }
0129:     if (action === "modernize") {
0130:         await handleModernize();
0131:         return;
0132:     }
0133:     if (action === "apply") {
0134:         applyOutputToEditor();
0135:     }
0136: }
0137:
0138: async function handleAnalyze() {
0139:     const code = codeInput.value;
0140:     if (!code.trim()) {
0141:         showToast("Paste legacy Python before analyzing", "error");
0142:         return;
0143:     }
0144:
0145:     state.originalCode = code;
0146:     state.newCode = "";
0147:     state.diff = "";
0148:     state.report = "";
0149:     state.runId = "";
0150:     state.validation = null;

```

Excerpt 3: UI State Machine Part 2

File: /Users/rutwikbhondave/Desktop/Helix AI Project/static/js/app.js

Simple explanation: this block handles frontend state machine and user interaction handling. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=0, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0261:         downloadReport.classList.remove("hidden");
0262:     }
0263:
0264:     setStepState("validate", ["analyze", "review", "modernize"]);
0265:     setMode(state.analysis?.mode || "Safe");
0266:     setStatus(state.validation?.success ? "Validated" : "Needs Review");
0267:     setFocus(
0268:         state.validation?.success
0269:         ? "Modernization finished. Review the diff, then apply the updated code if you want."
0270:         : "Modernization finished with warnings. Review the diff and report before trusting."
0271:     );
0272:     setPrimaryAction(state.newCode && state.newCode !== state.originalCode ? "apply" : "analyze");
0273:
0274:     if (state.runId) {
0275:         log(`Saved run ${state.runId}.`, "success");
0276:         loadRunHistory();
0277:     }
0278:
0279:     showToast(state.validation?.success ? "Modernization completed" : "Modernization finished with warnings");
0280: } catch (error) {
0281:     setMode("Error");
0282:     setStatus("Modernization Failed");
0283:     setFocus("Modernization failed. Check the advanced log and try again.");
0284:     setPrimaryAction("modernize");
0285:     log(error.message || "Modernization failed.", "error");
0286:     showToast("Modernization failed", "error");
0287: }
0288: }
0289:
0290: function applyOutputToEditor() {
0291:     if (!state.newCode) {
0292:         return;
0293:     }
0294:     codeInput.value = state.newCode;
0295:     updateLineNumbers();
0296:     syncScroll();
0297:     setPrimaryAction("analyze");
0298:     setFocus("Changes were applied to the editor. You can analyze again or continue editing.");
0299:     showToast("Updated code applied to editor", "success");
0300: }
0301:
0302: function handleStepNavigation(step) {
0303:     if (step === "analyze") {
0304:         codeInput.focus();
```

```

0305:         codeInput.scrollIntoView({ behavior: "smooth", block: "center" });
0306:         showToast("Editor focused", "info");
0307:         return;
0308:     }
0309:
0310:     if (step === "review") {
0311:         if (!state.analysis) {
0312:             showToast("Analyze the code first", "error");
0313:             return;
0314:         }
0315:         activateTab("review");
0316:         showToast("Review opened", "success");
0317:         return;
0318:     }
0319:
0320:     if (step === "modernize") {
0321:         if (!state.analysis) {
0322:             showToast("Analyze the code first", "error");
0323:             return;
0324:         }
0325:         activateTab("review");
0326:         runBtn.scrollIntoView({ behavior: "smooth", block: "center" });
0327:         runBtn.focus();
0328:         showToast(state.primaryAction === "modernize" ? "Ready to modernize" : "Review the findings", "success");
0329:         return;
0330:     }
0331:
0332:     if (step === "validate") {
0333:         if (!state.diff && !state.report && !state.newCode) {
0334:             showToast("Run modernization to review the result", "error");
0335:             return;
0336:         }
0337:         const targetTab = state.diff ? "diff" : (state.newCode ? "output" : "report");
0338:         activateTab(targetTab);
0339:         showToast("Validation view opened", "success");
0340:     }
0341: }
0342:
0343: async function fetchModelStatus() {
0344:     try {
0345:         const response = await fetch("/model-status");
0346:         const data = await response.json();
0347:         state.ml = data;
0348:         renderMlSummary(data);
0349:     } catch (error) {
0350:         renderMlSummary(null);
0351:     }
0352: }
0353:
0354: async function loadRunHistory() {
0355:     try {
0356:         const response = await fetch("/runs?limit=6");
0357:         const data = await response.json();
0358:         const runs = data.runs || [];
0359:
0360:         if (!runs.length) {
0361:             runHistory.innerHTML = `<div class="rounded-xl border border-dashed border-white/10 bg-white p-4">
0362:                 No runs found.
0363:             </div>`;
0364:             return;
0365:         }
0366:         runHistory.innerHTML = runs.map((run) => `
0367:             <button data-run-id="${escapeHtml(run.run_id)}" class="w-full rounded-xl border border-gray-200 p-4">
0368:                 <div class="flex items-center justify-between gap-3">
0369:                     <div class="font-mono text-[11px] text-zinc-300">${escapeHtml(run.run_id.slice(0, 10))}</div>
0370:                     <span class="text-[10px] uppercase tracking-[0.18em] ${run.validation_success ? 'text-green-500' : 'text-red-500'}">${run.validation_success ? 'Success' : 'Error'}</span>
0371:                 </div>
0372:             </button>
0373:         `);
0374:     } catch (error) {
0375:         console.error("Error loading run history:", error);
0376:     }
0377: }

```